

Data-parallel training (naive)

cs.cmu.edu/~muli/file/parameter_server_osdi14.pdf

Host

CPU

model



Devices

.cuda()

GPU1



Data-parallel training (naive)

cs.cmu.edu/~muli/file/parameter_server_osdi14.pdf

Host

CPU

model

θ

batch

x

...

Devices

.cuda()

.cuda()

GPU1

θ

x

$\Rightarrow$

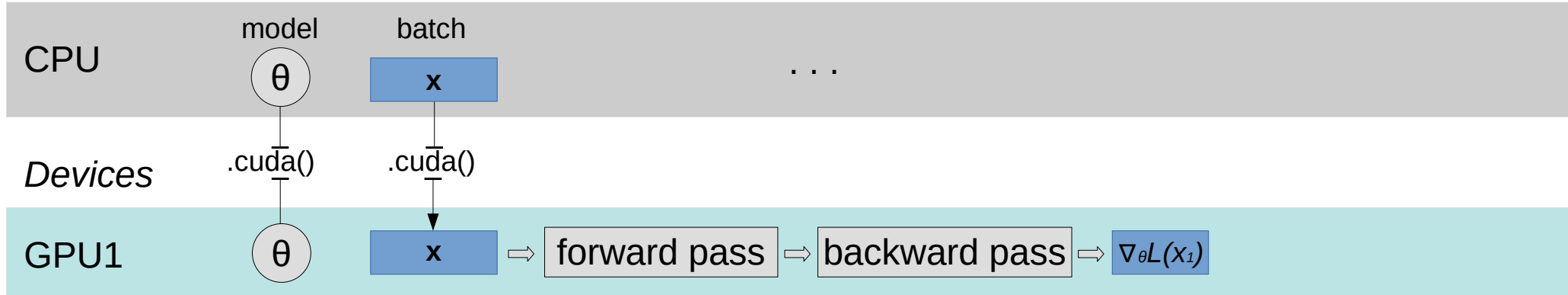
forward pass

$\Rightarrow$

backward pass

$\Rightarrow$

$\nabla_{\theta} L(x_1)$



Data-parallel training (naive)

cs.cmu.edu/~muli/file/parameter_server_osdi14.pdf

Host

CPU

model

θ

batch

x

...

prepare next batch

Devices

.cuda()

.cuda()

new model

GPU1

θ

x

$\Rightarrow$

forward pass

$\Rightarrow$

backward pass

$\Rightarrow$

$\nabla_{\theta} L(x_1)$

$\Rightarrow$

step

$\Rightarrow$

θ

Data-parallel training (naive)

cs.cmu.edu/~muli/file/parameter_server_osdi14.pdf

Host

CPU

model

θ

Devices

replicate

GPU1

θ

GPU2

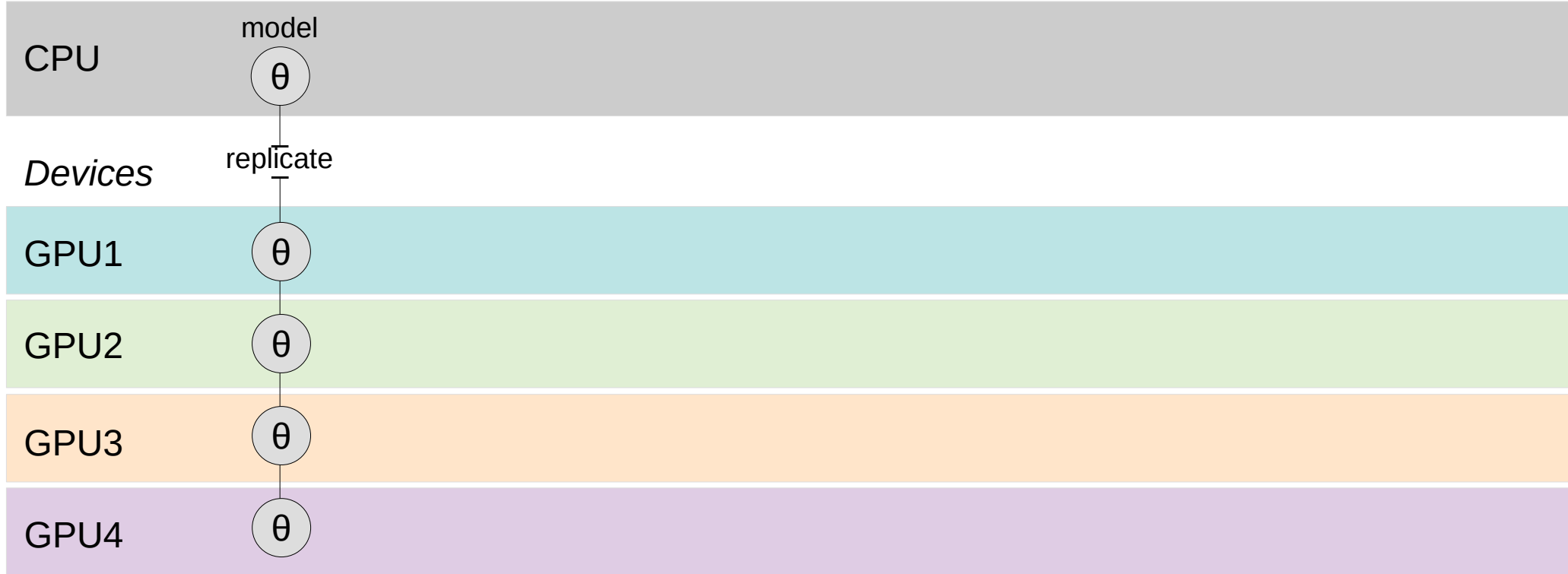
θ

GPU3

θ

GPU4

θ



Data-parallel training (naive)

cs.cmu.edu/~muli/file/parameter_server_osdi14.pdf

Host



Devices

replicate

scatter

GPU1



x_1

GPU2



x_2

GPU3

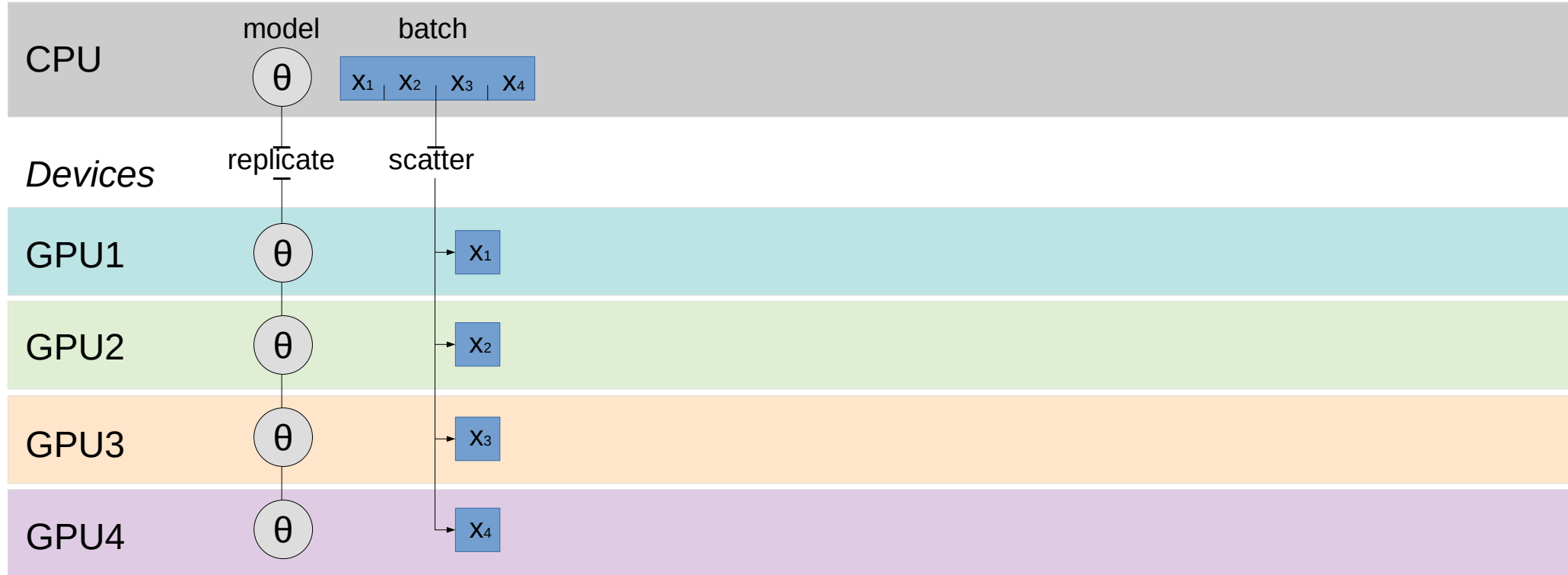


x_3

GPU4



x_4



Data-parallel training (naive)

cs.cmu.edu/~muli/file/parameter_server_osdi14.pdf

Host

CPU

model

θ

batch

x_1 x_2 x_3 x_4

...

Devices

replicate

scatter

GPU1

θ

$x_1 \Rightarrow$ forward pass $\Rightarrow$ backward pass $\Rightarrow \nabla_{\theta} L(x_1)$

GPU2

θ

$x_2 \Rightarrow$ forward pass $\Rightarrow$ backward pass $\Rightarrow \nabla_{\theta} L(x_2)$

GPU3

θ

$x_3 \Rightarrow$ forward pass $\Rightarrow$ backward pass $\Rightarrow \nabla_{\theta} L(x_3)$

GPU4

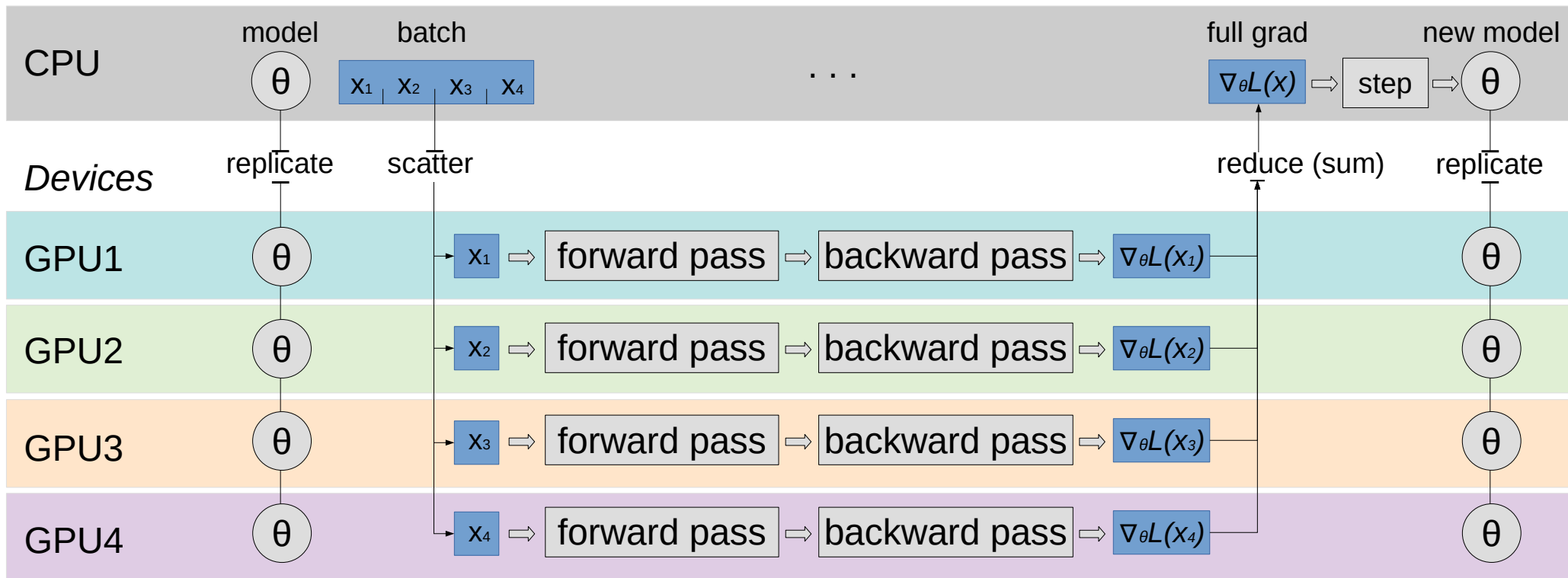
θ

$x_4 \Rightarrow$ forward pass $\Rightarrow$ backward pass $\Rightarrow \nabla_{\theta} L(x_4)$

Data-parallel training (naive)

cs.cmu.edu/~muli/file/parameter_server_osdi14.pdf

Host

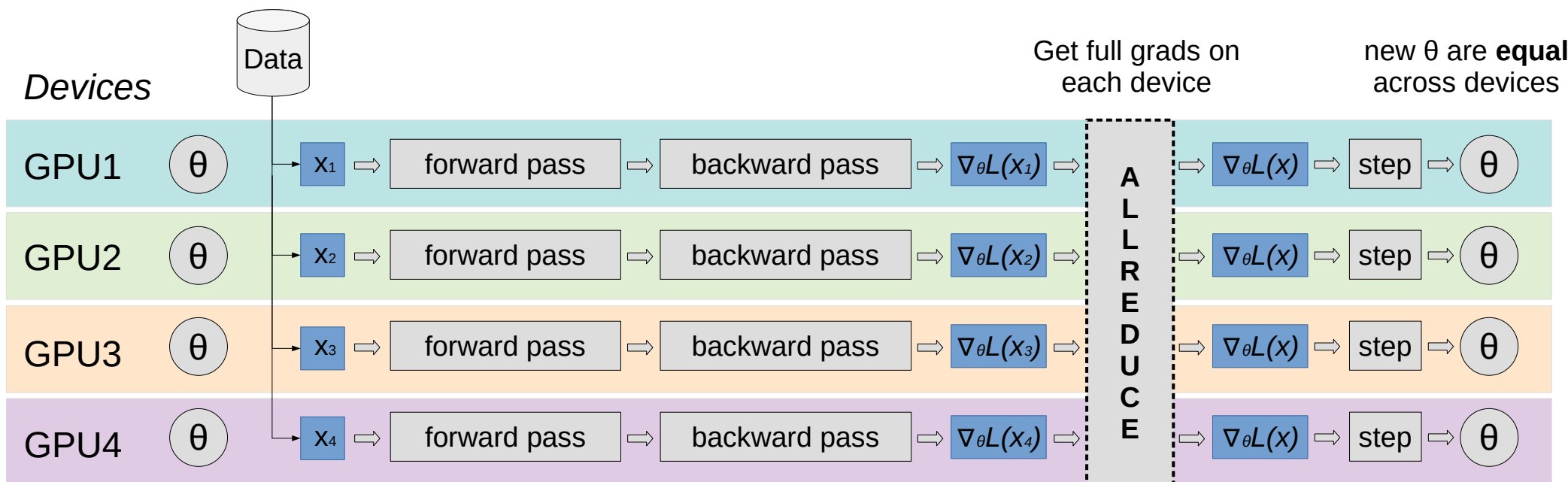


All-Reduce data parallel

arxiv.org/abs/1706.02677

Idea: get rid of the host, each gpu runs its own computation

Q: why will weights be equal after such step?

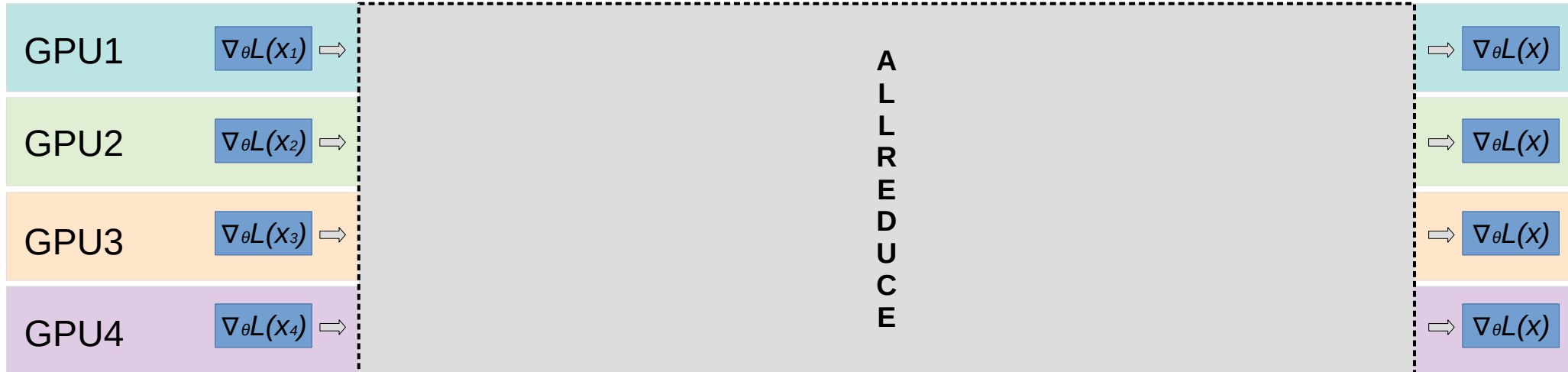


Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

Devices



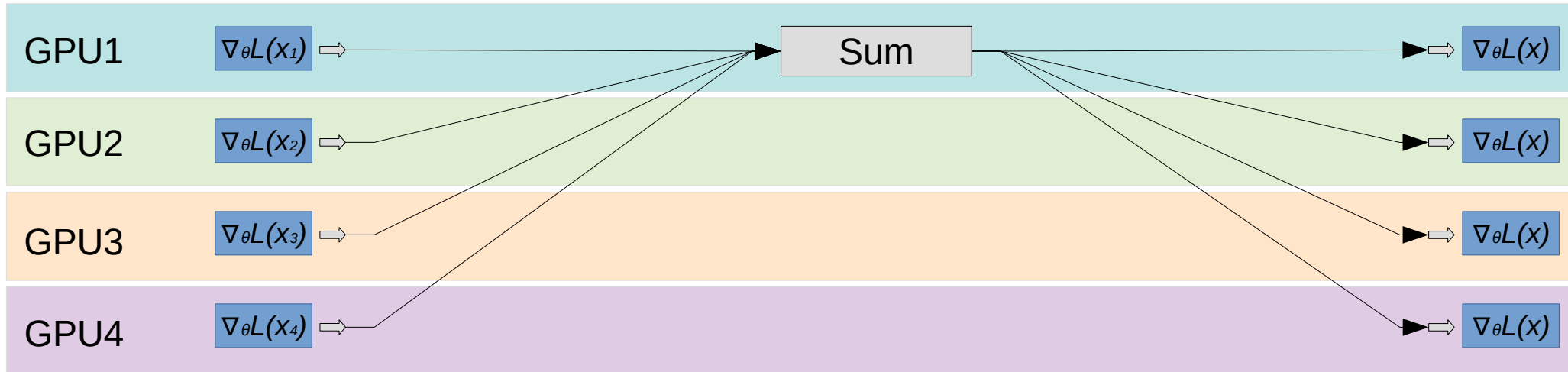
Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

Naive implementation

Devices



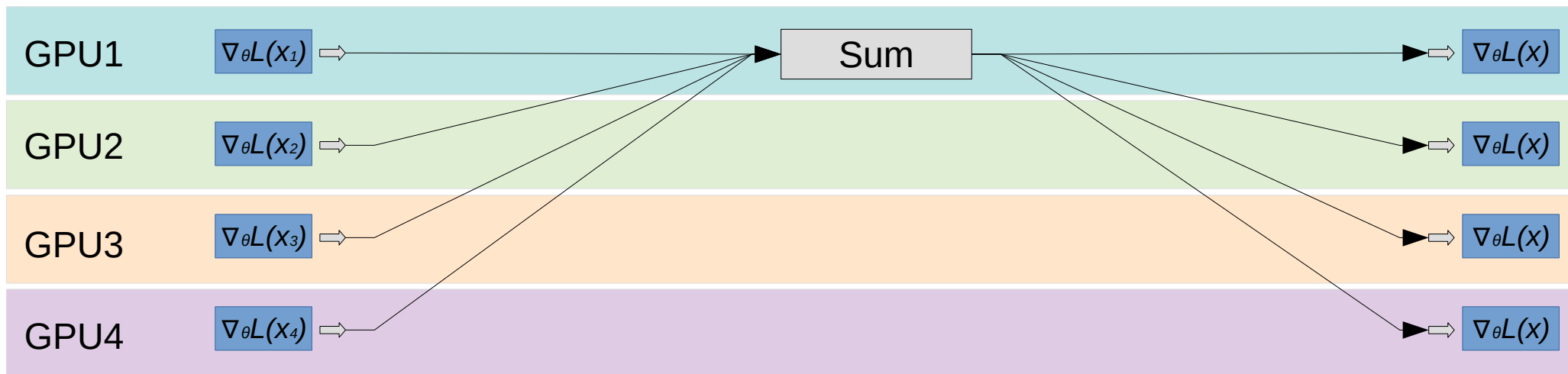
Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

Q: Can we do better?

Devices



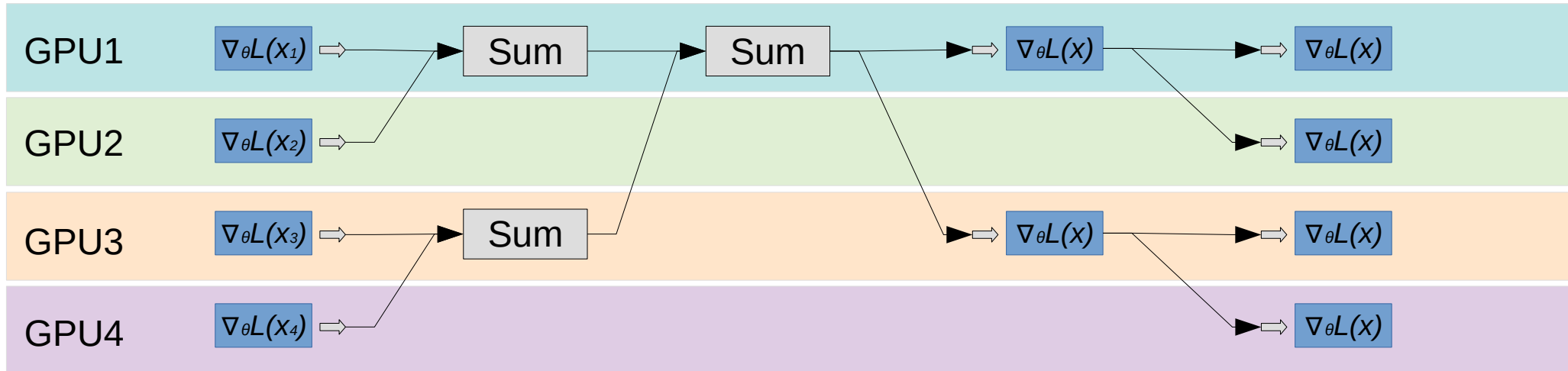
Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

Tree-allreduce

Devices



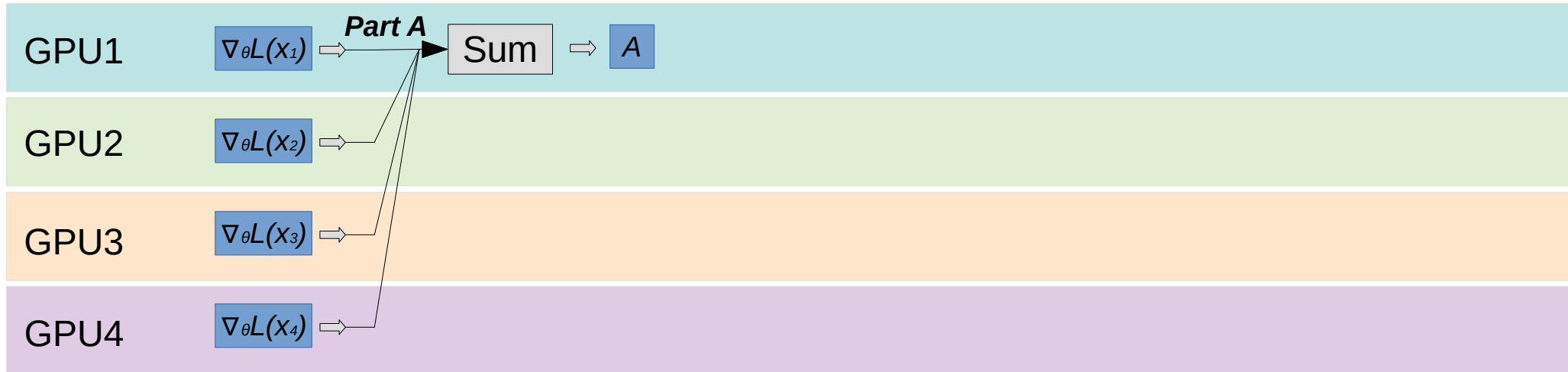
Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

Butterfly-allreduce – split data into chunks (ABCD)

Devices



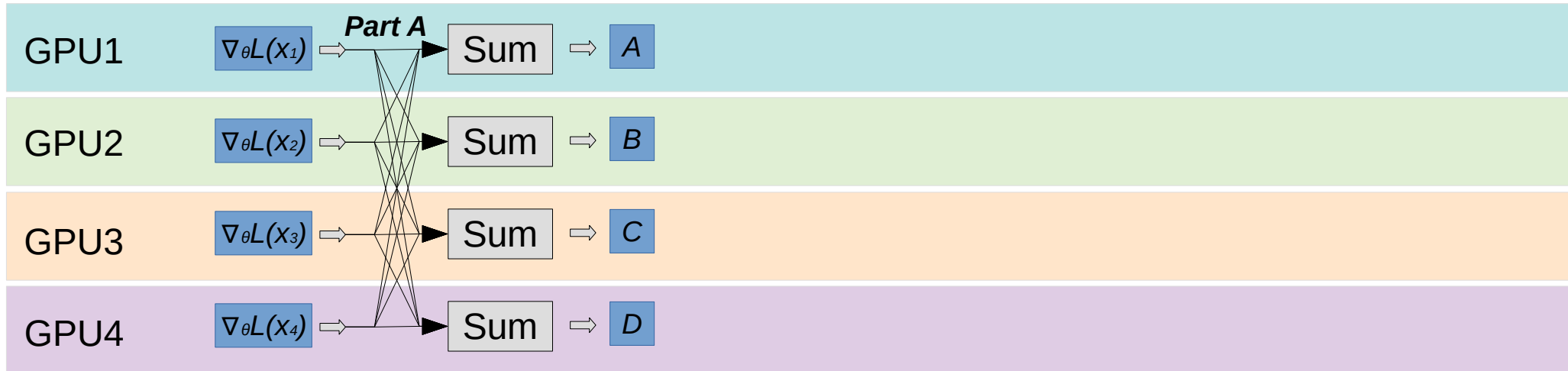
Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

Butterfly-allreduce – split data into chunks (ABCD)

Devices



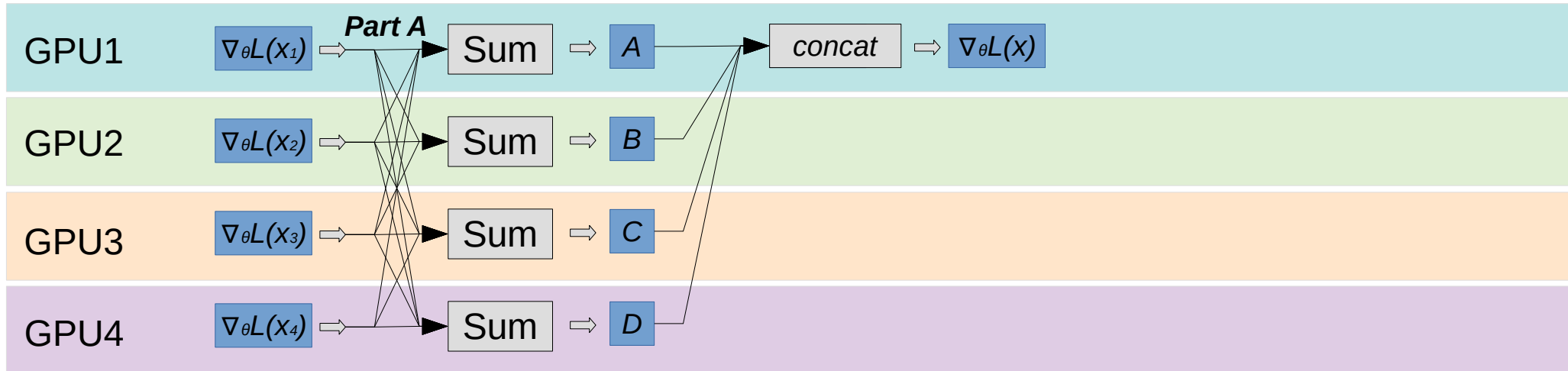
Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

Butterfly-allreduce – split data into chunks (ABCD)

Devices



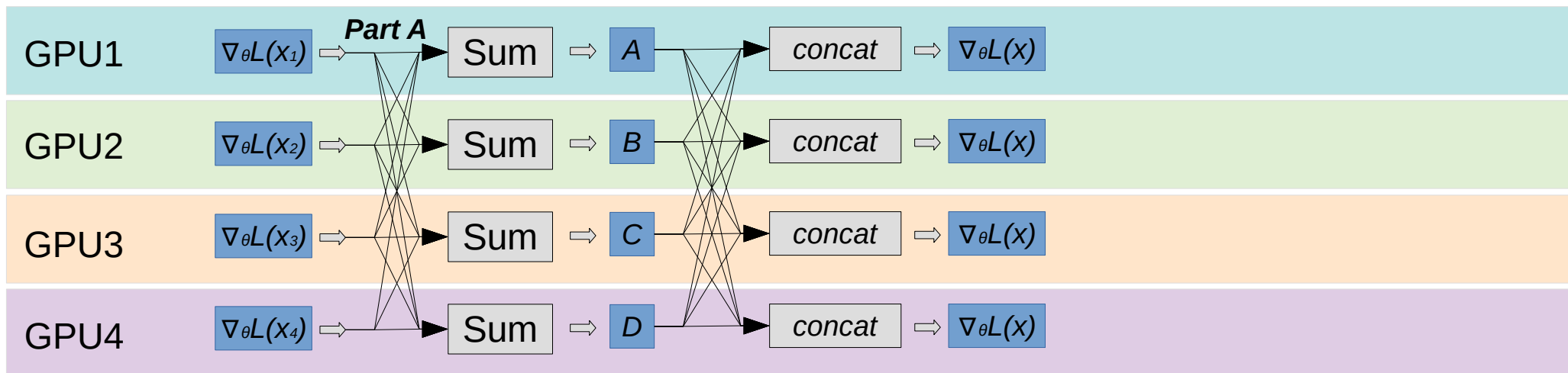
Faster allreduce

Input: each device has its own vector

Output: each device gets a sum of all vectors

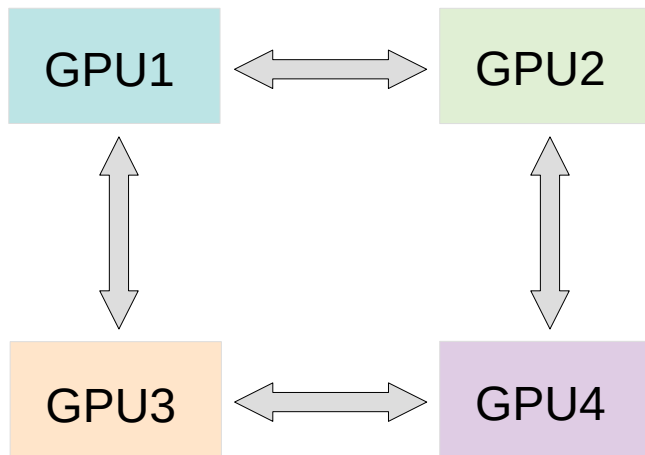
Ring-allreduce – split data into chunks (ABCD)

Devices



Ring allreduce

Bonus quest: you can only send data between **adjacent** gpus



Ring topology



*Image: **graphcore** IPU server*

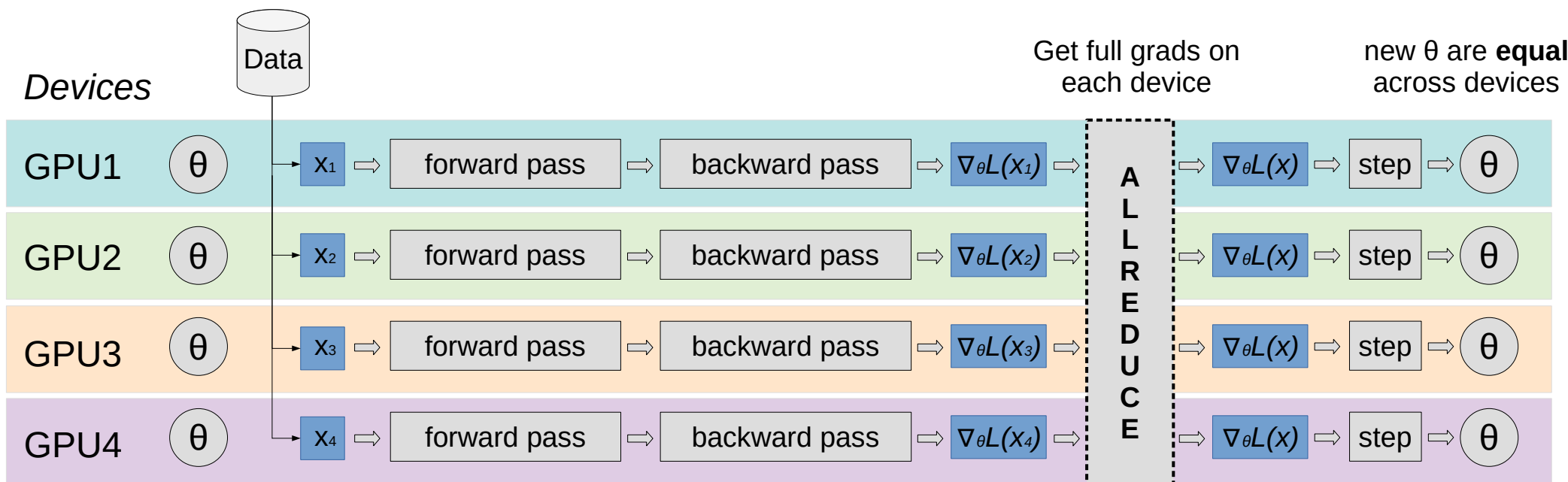
Answer & more: tinyurl.com/ring-allreduce-blog

All-Reduce data parallel

arxiv.org/abs/1706.02677

Idea: get rid of the host, each gpu runs its own computation

Q: why will weights be equal after such step?



</Data-parallel>

- + easy to implement
 - + can scale to 100s of gpus
 - + can be fault-tolerant
 - model must fit in 1 gpu
 - large batches aren't always good for generalization
-
- 2-4 GPUs & no time – naive data parallel tinyurl.com/torch-data-parallel
 - 4+ GPUs or multiple hosts – distributed (allreduce) github.com/horovod/horovod
 - Intro to pytorch distributed: tinyurl.com/distributed-dp or in **15 minutes!**
 - Somewhat faulty GPU/network: synchronous data parallel + drop stragglers
 - Very faulty or uneven resources: asynchronous data parallel (more later)
 - Efficient training with large batches: LAMB <https://arxiv.org/abs/1904.00962>
 - Dynamically adding or removing resources: <https://tinyurl.com/torch-elastic>

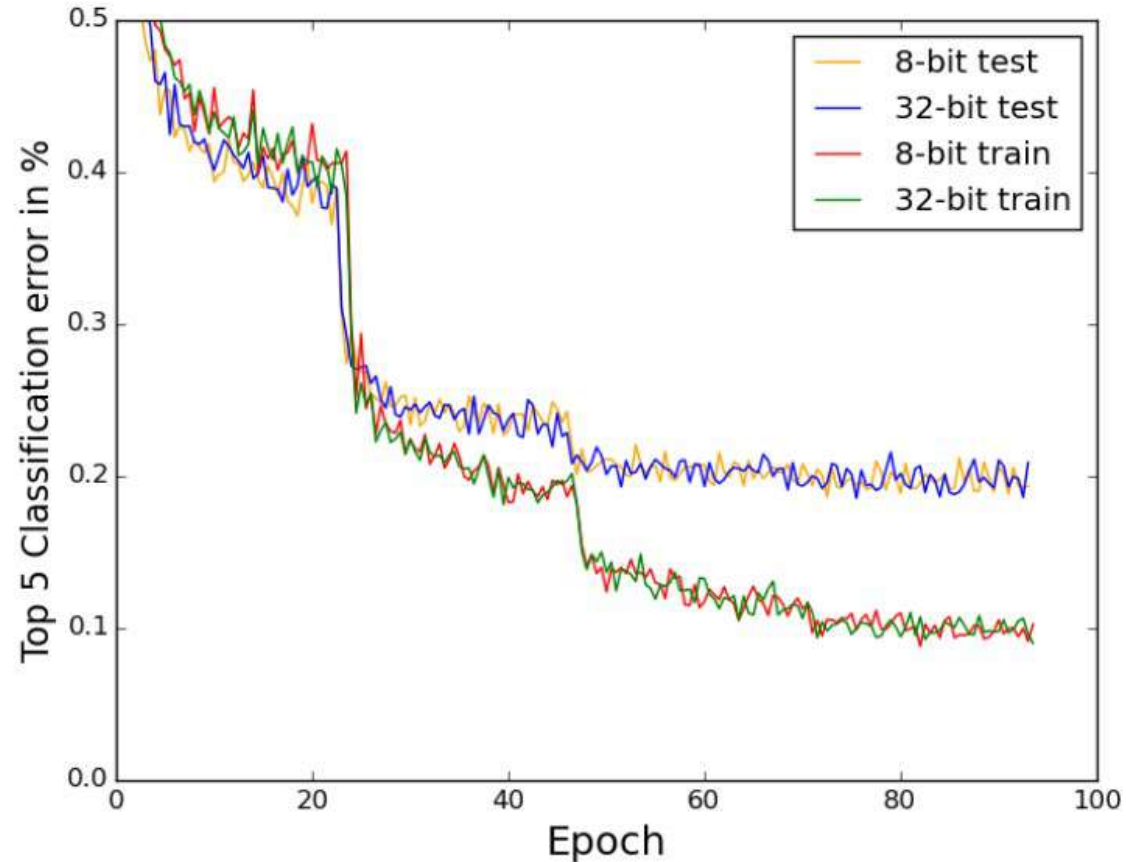
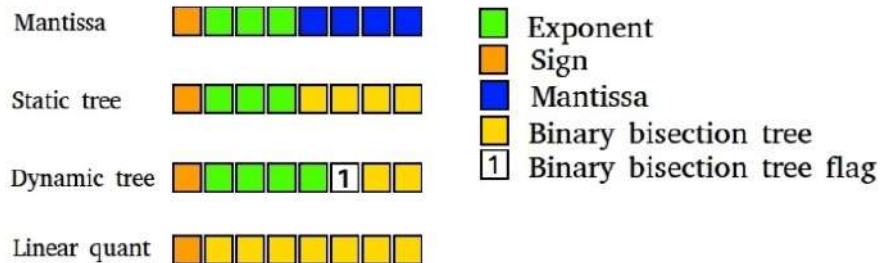
Quantized communication

<https://arxiv.org/abs/1511.04561>

TL;DR

- send data in 8-bit
- all computations in 32-bit
- choose best data format

PROFIT: same quality as float16



Can we compress further?
without losing quality

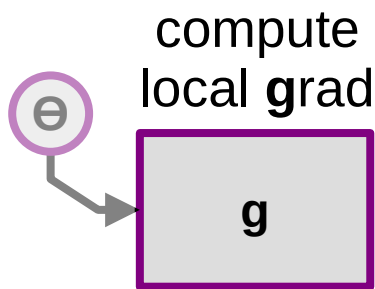
Error Feedback + PowerSGD

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/1905.13727> - PowerSGD

TL;DR - use extreme compression, e.g. 1-bit or top-5% gradients

- if you lose something in compression, **reuse it on the next step**



time flows left to right →

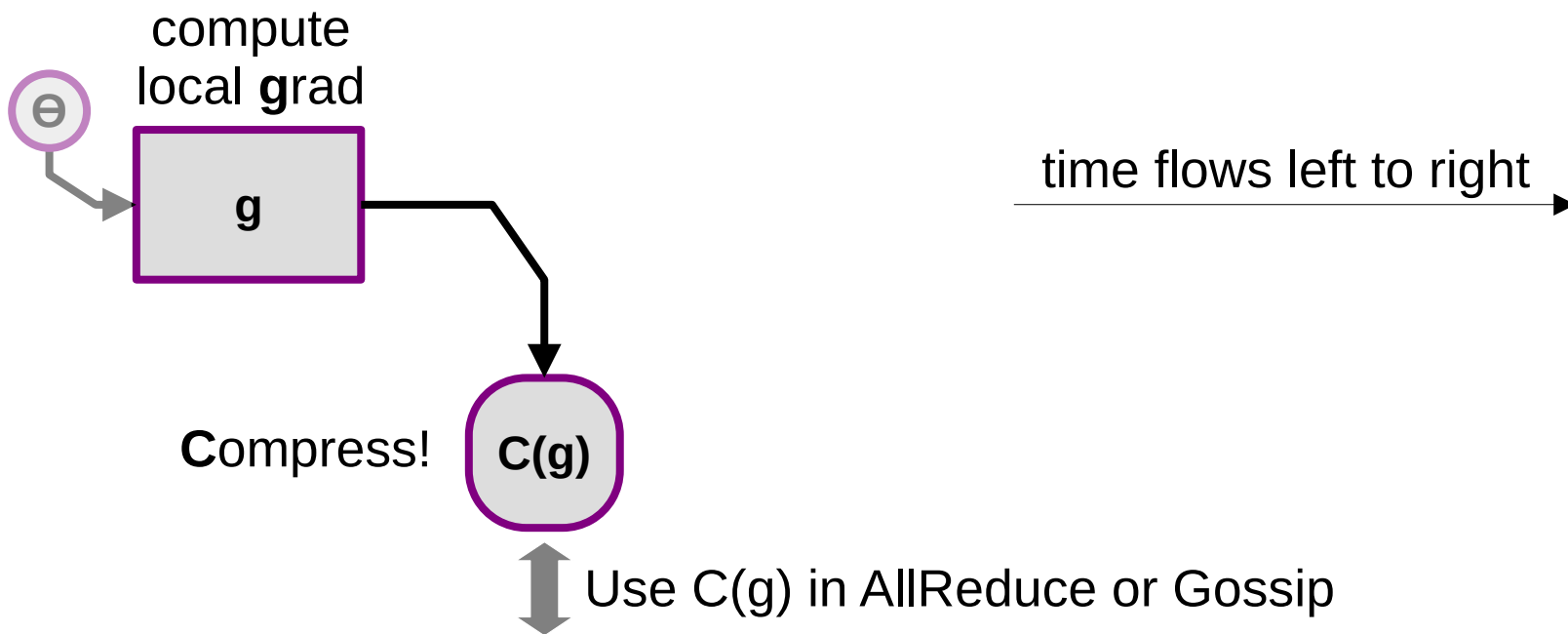
Error Feedback + PowerSGD

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/1905.13727> - PowerSGD

TL;DR - use extreme compression, e.g. 1-bit or top-5% gradients

- if you lose something in compression, **reuse it on the next step**



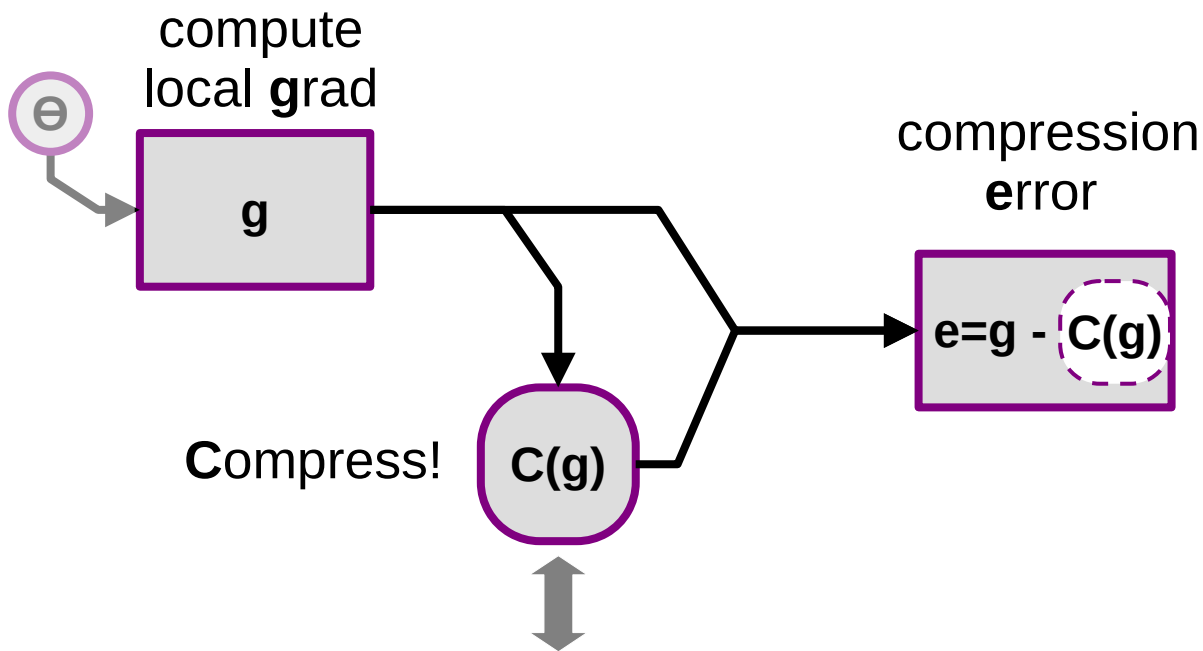
Error Feedback + PowerSGD

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/1905.13727> - PowerSGD

TL;DR - use extreme compression, e.g. 1-bit or top-5% gradients

- if you lose something in compression, **reuse it on the next step**



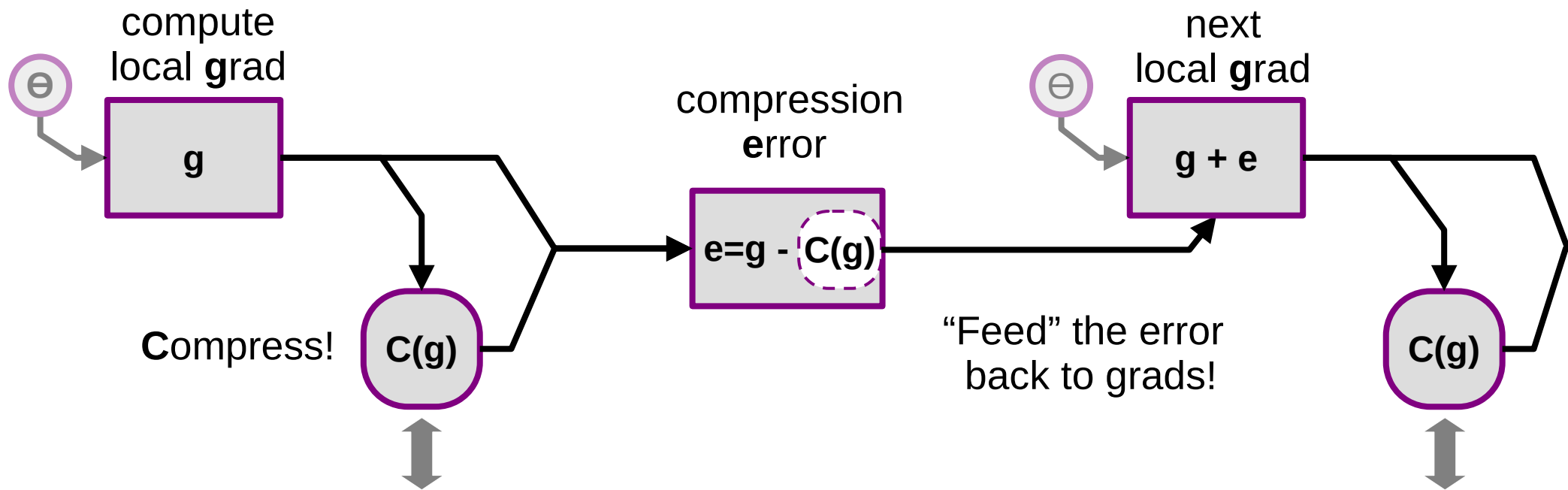
Error Feedback + PowerSGD

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/1905.13727> - PowerSGD

TL;DR - use extreme compression, e.g. 1-bit or top-5% gradients

- if you lose something in compression, **reuse it on the next step**



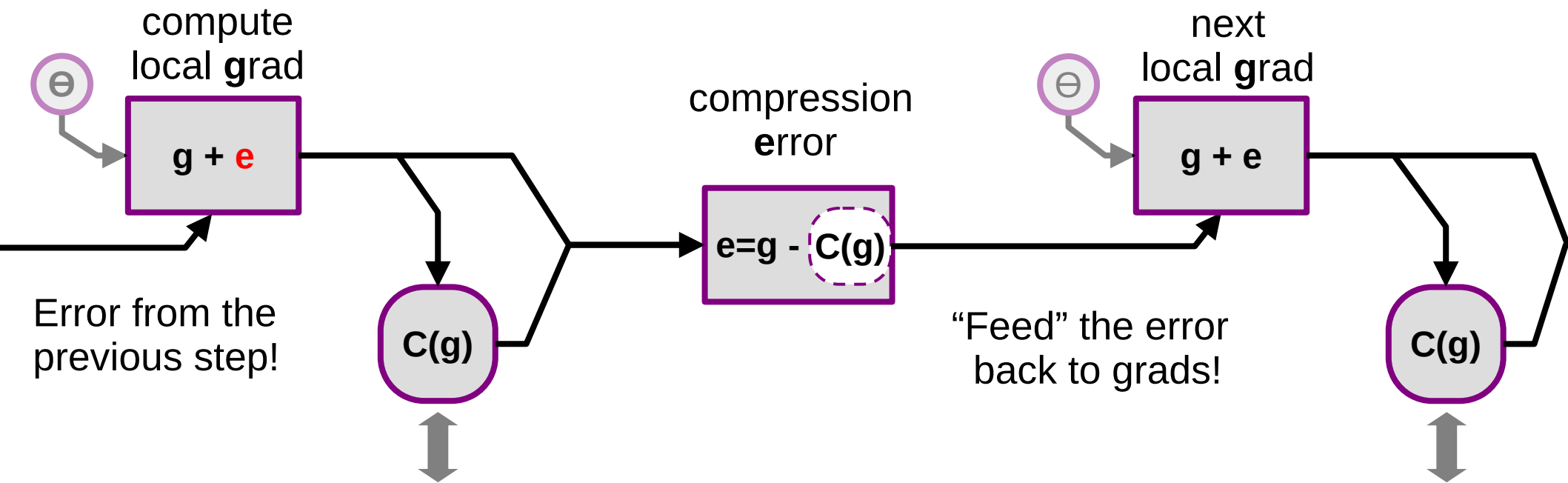
Error Feedback + PowerSGD

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/1905.13727> - PowerSGD

TL;DR - use extreme compression, e.g. 1-bit or top-5% gradients

- if you lose something in compression, **reuse it on the next step**



Error Feedback + PowerSGD

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/1905.13727> - PowerSGD

```
1: hyperparameters: learning rate  $\gamma$ , momentum parameter  $\lambda$ 
2: initialize model parameters  $\mathbf{x} \in \mathbb{R}^d$ , momentum  $\mathbf{m} \leftarrow \mathbf{0} \in \mathbb{R}^d$ , replicated across workers
3: at each worker  $w = 1, \dots, W$  do
4:   initialize memory  $\mathbf{e}_w \leftarrow \mathbf{0} \in \mathbb{R}^d$ 
5:   for each iterate  $t = 0, \dots$  do
6:     Compute a stochastic gradient  $\mathbf{g}_w \in \mathbb{R}^d$ .
7:      $\Delta_w \leftarrow \mathbf{g}_w + \mathbf{e}_w$  ▷ Incorporate error-feedback into update
8:      $\mathcal{C}(\Delta_w) \leftarrow \text{COMPRESS}(\Delta_w)$ 
9:      $\mathbf{e}_w \leftarrow \Delta_w - \text{DECOMPRESS}(\mathcal{C}(\Delta_w))$  ▷ Memorize local errors
10:     $\mathcal{C}(\Delta) \leftarrow \text{AGGREGATE}(\mathcal{C}(\Delta_1), \dots, \mathcal{C}(\Delta_W))$  ▷ Exchange gradients
11:     $\Delta' \leftarrow \text{DECOMPRESS}(\mathcal{C}(\Delta))$  ▷ Reconstruct an update  $\in \mathbb{R}^d$ 
12:     $\mathbf{m} \leftarrow \lambda \mathbf{m} + \Delta'$ 
13:     $\mathbf{x} \leftarrow \mathbf{x} - \gamma (\Delta' + \mathbf{m})$ 
14:  end for
15: end at
```

PowerSGD: low-rank approx grads + Error Feedback

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/1905.13727> - PowerSGD

- 1: The update vector Δ_w is treated as a list of tensors corresponding to individual model parameters. Vector-shaped parameters (biases) are aggregated uncompressed. Other parameters are reshaped into matrices. The functions below operate on such matrices independently. For each matrix $M \in \mathbb{R}^{n \times m}$, a corresponding $Q \in \mathbb{R}^{m \times r}$ is initialized from an i.i.d. standard normal distribution.
- 2: **function** COMPRESS+AGGREGATE(update matrix $M \in \mathbb{R}^{n \times m}$, previous $Q \in \mathbb{R}^{m \times r}$)
- 3: $P \leftarrow MQ$
- 4: $P \leftarrow \text{ALL REDUCE MEAN}(P)$ ▷ Now, $P = \frac{1}{W}(M_1 + \dots + M_W)Q$
- 5: $\hat{P} \leftarrow \text{ORTHOGONALIZE}(P)$ ▷ Orthonormal columns
- 6: $Q \leftarrow M^\top \hat{P}$
- 7: $Q \leftarrow \text{ALL REDUCE MEAN}(Q)$ ▷ Now, $Q = \frac{1}{W}(M_1 + \dots + M_W)^\top \hat{P}$
- 8: **return** the compressed representation $(\hat{P}, Q)$.
- 9: **end function**
- 10: **function** DECOMPRESS($\hat{P} \in \mathbb{R}^{n \times r}$, $Q \in \mathbb{R}^{m \times r}$)
- 11: **return** $\hat{P}Q^\top$
- 12: **end function**

Read More: gradient compression

<https://arxiv.org/abs/1901.09847> - EF theory

<https://arxiv.org/abs/2106.05203> - better EF'21

<https://arxiv.org/abs/1905.13727> - PowerSGD

<https://arxiv.org/abs/2110.03294> - more EF'21

```
1 import torch.distributed.algorithms.ddp_comm_hooks.powerSGD_hook as powerSGD
2
3 ddp_model = nn.parallel.DistributedDataParallel(
4     module=model,
5     device_ids=[rank],
6 )
7
8 state = PowerSGD.PowerSGDState(
9     process_group=process_group,
10    matrix_approximation_rank=1,
11    start_powerSGD_iter=1_000,
12 )
13 ddp_model.register_comm_hook(state, PowerSGD.powerSGD_hook)
```